Kolkata 700 091

Name of Student :

1. Sayantan Ghosh
2. Sayantan Mondal
3. Soham Chakraborty
4. Soham Chatterjee
5. Soumyadeep Ghosh

1. Linear Regression Model

```
!pip install -q -U keras-nlp
!pip install -q keras-tuner
!pip install -q -U keras-core keras
!pip install -q kagglehub
!pip install -q tensorflow-text
!pip install -q tensorflow-hub
!pip install -q tensorflow-cpu
```

```
!git clone https://github.com/Soham-Chakraborty/Sentiment-Analysis-Using-Machine-Learning-Models.git
```

```
Cloning into 'Sentiment-Analysis-Using-Machine-Learning-Models'...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressed objects: 100% (6/6), done.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (7/7), done.
```

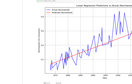
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor
from keras.models import Sequential
from keras.layers import Dense, Dropout, BatchNormalization
from keras.callbacks import EarlyStopping, ReduceLROnPlateau
from keras.optimizers import Adam
from keras.regularizers import l1, l2, l1_l2
from keras_tuner import RandomSearch
from keras_tuner.engine.hyperparameters import HyperParameters
from keras.utils import to_categorical
from keras.models import load_model
from keras.callbacks import ModelCheckpoint
from google.colab.drive import mount
from google.colab import files
from tqdm.keras import TqdmCallback
from tqdm.notebook import tqdm
from scipy.stats import pearsonr, spearmanr, kendalltau
from sklearn.feature_selection import mutual_info_regression, SelectKBest, f_regression
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin
from joblib import dump, load
import os
import glob
import warnings
warnings.filterwarnings('ignore')
```

```
mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Load the data
data = pd.read_csv('/content/Sentiment-Analysis-Using-Machine-Learning-Models/amazon_alexa.csv')

# Preprocessing
le = LabelEncoder()
data['variation'] = le.fit_transform(data['variation'])

# Feature engineering
data['length'] = data['verified_reviews'].str.len()
data['word_count'] = data['verified_reviews'].str.split().str.len()

# Prepare features and target
X = data[['variation', 'length', 'word_count']]
y = data['rating']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create and train the model
model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Predicting on the test set
y_pred = model.predict(X_test_scaled)

# Evaluation metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R-squared:", r2)

# Plot actual vs predicted ratings
plt.figure(figure_size=(10,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Ratings')
plt.ylabel('Predicted Ratings')
plt.title('Linear Regression Predictions vs Actual Ratings')
plt.show()
```

```
Mean Squared Error: 1.1111111111111111
Root Mean Squared Error: 1.0540925533894598
R-squared: -0.0000000000000002
```




```
# Load the data
data = pd.read_csv('/content/Sentiment-Analysis-Using-Machine-Learning-Models/amazon_alexa.csv')

# Preprocessing
le = LabelEncoder()
data['variation'] = le.fit_transform(data['variation'])

# Feature engineering
data['length'] = data['verified_reviews'].str.len()
data['word_count'] = data['verified_reviews'].str.split().str.len()

# Prepare features and target
X = data[['variation', 'length', 'word_count']]
y = data['rating']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create and train the model
model = LGBMRegressor(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

# Predicting on the test set
y_pred = model.predict(X_test_scaled)

# Evaluation metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R-squared:", r2)

# Plot actual vs predicted ratings
plt.figure(figure_size=(10,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Ratings')
plt.ylabel('Predicted Ratings')
plt.title('LightGBM Predictions vs Actual Ratings')
plt.show
```

```
[LightGBM] [Warning] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000279 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1275
[LightGBM] [Info] Number of data points in the train set: 2553, of which 0 were forced bins.
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.000000 -> initscore=-13.815511
```